Reviewer Pack — Minimal Tropical Symmetry Length and DPLL Proof Path Length …

Entry 5e859aa7a62b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 06:58:59 UTC

Minimal Tropical Symmetry Length and DPLL Proof Path Length Lower Bound

Entry ID: `5e859aa7a62b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 06:58:59 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

> For every boolean satisfiability instance φ with n variables, the minimal tropical symmetry length (msl(φ)) of its associated Tseitin formula is linearly correlated with its DPLL proof path length l(φ), such that msl(φ) = Ω(l(φ)).

Rationale (proposer's reasoning):

> Tropical geometry provides a framework for studying discrete structures in a way that can reveal symmetries and patterns. If the minimal tropical symmetry length of a Tseitin formula correlates with the DPLL proof path length, it suggests that symmetrical properties of the formula may be exploited to design more efficient algorithms or to construct lower bounds on the proof complexity.

Taxonomy category: `TROPICAL GEOMETRY × BOOLEAN SATISFIABILITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9acd891d3e3c3ff9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of the generated instances with n ≤ 40 variables, the linear regression coefficient between msl(φ) and l(φ) exceeds 1.0, and no instance has a msl(φ) less than half its corresponding l(φ).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.5s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_formula(n):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(2**n - 1):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [f'-{v}' if v.startswith('-') else f'-{v}' for v in clause]
            clauses.append(' or '.join(clause))
        return ' and '.join(clauses)

    def tseitin_formula(formula):
        n = len(formula.split())
        literals = set()
        for i in range(n):
            literals.add(f'x{i}')
            literals.add(f'-x{i}')

        clauses = []
        for literal in literals:
            if literal.startswith('-'):
                continue
            clause = [f'{literal}', f'-{literal}']
            clauses.append(' or '.join(clause))

        return ' and '.join(clauses)

    def dpll(formula):
        # Simplified DPLL solver
        variables = set()
        for literal in formula.split():
            if literal.startswith('-'):
```

```
                continue
            variables.add(literal)

    stack = []
    assignment = {}
    for variable in variables:
        stack.append((variable, True))
        stack.append((variable, False))

    def backtrack():
        while stack:
            literal, value = stack.pop()
            if literal not in assignment:
                assignment[literal] = value
                break
        else:
            return None

        new_formula = formula.replace(literal, str(value)).replace(f'-{literal}', str(not valu
        result = dpll(new_formula)
        if result is not None:
            return result
        del assignment[literal]

        new_formula = formula.replace(literal, str(not value)).replace(f'-{literal}', str(valu
        result = dpll(new_formula)
        if result is not None:
            return result
        del assignment[literal]

    return backtrack()

def minimal_tropical_symmetry_length(formula):
    # Simplified tropical symmetry length calculation
    n = len(formula.split())
    return n

instances_tested = 0
msl_sum = 0
l_sum = 0
counterexample_found = False

for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_random_boolean_formula(n)
    tseitin = tseitin_formula(formula)

    msl = minimal_tropical_symmetry_length(tseitin)
    l = dpll(formula)

    if l is None:
        continue

    instances_tested += 1
    msl_sum += msl
    l_sum += l
```

```
            if msl < l / 2:
                counterexample_found = True

    metric_value = msl_sum / instances_tested if instances_tested > 0 else 0
    conjecture_holds = not counterexample_found and metric_value >= 1.0
    n_max = max([5, 10, 15, 20, 30, 40])

    return {
        "metric_name": "minimal_tropical_symmetry_length",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": "Counterexample found" if counterexample_found else ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"Counterexample found\" first_failing_seed={firs
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21639
2	propose	ollama_remote	glm4:latest	0	0	15786
3	preregistration	ollama_remote	glm4:latest	0	0	10353
4	novelty	ollama_remote	glm4:latest	0	0	8866
5	novelty	ollama_remote	glm4:latest	0	0	8691
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21640
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11036
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10653
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13162
10	judge	ollama_remote	glm4:latest	0	0	36725

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 158550 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5e859aa7a62b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5e859aa7a62b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5e859aa7a62b.tar.gz` (if generated)